

# Projet\_apprentissage

May 25, 2025

## 1 Reinforcement Learning Project : N7Craft

## 2 Training a Minecraft Jump Agent with Mineflayer & RL

### A Gym-style Env, PPO Training Loop, and Evaluation on Custom Jump Maps

Briefly outline the goal of this notebook and the high-level flow:

- Connect Mineflayer bot to a custom jump map server
- Wrap it in a Gym environment
- Train an RL agent (e.g. PPO)
- Evaluate performance

### 2.1 Environment Setup & Imports

Launch the Minecraft Server (version 1.19.4 if you want to connect to it as a player)

Install and import all required libraries: 1. `pip install gym mineflayer vec3 stable-baselines3 torch matplotlib`  
2. Import Python modules for Gym, Mineflayer, NumPy, etc.

```
[1]: java -Xms1G -Xmx2G -jar minecraft-server\paper-1.19.4-550.jar
```

```
Cell In[1], line 1
```

```
java -Xms1G -Xmx2G -jar minecraft-server\paper-1.19.4-550.jar
```

```
SyntaxError: unexpected character after line continuation character
```

```
[1]: # Use `n` to install nodejs 18, if it's not already installed:
#!curl -fsSL https://raw.githubusercontent.com/tj/n/master/bin/n | bash -s lts
↵> /dev/null
# Now write the Node.js and Python version to the console
!node --version
!python --version
```

v22.13.1

Python 3.13.1

```
[2]: !pip install javascript
```

Requirement already satisfied: javascript in  
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages  
(1!1.2.2)

```
[3]: !pip install gym vec3 javascript stable-baselines3 torch matplotlib
```

Requirement already satisfied: gym in  
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages  
(0.26.2)  
Requirement already satisfied: vec3 in  
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (0.1.0)  
Requirement already satisfied: javascript in  
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages  
(1!1.2.2)  
Requirement already satisfied: stable-baselines3 in  
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (2.6.0)  
Requirement already satisfied: torch in  
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (2.7.0)  
Requirement already satisfied: matplotlib in  
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages  
(3.10.0)  
Requirement already satisfied: numpy>=1.18.0 in  
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from  
gym) (2.2.2)  
Requirement already satisfied: cloudpickle>=1.2.0 in  
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from  
gym) (3.1.1)  
Requirement already satisfied: gym\_notices>=0.0.4 in  
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from  
gym) (0.0.8)  
Requirement already satisfied: gymnasium<1.2.0,>=0.29.1 in  
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from  
stable-baselines3) (1.1.1)  
Requirement already satisfied: pandas in  
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from  
stable-baselines3) (2.2.3)  
Requirement already satisfied: filelock in  
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from  
torch) (3.17.0)  
Requirement already satisfied: typing-extensions>=4.10.0 in  
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from  
torch) (4.12.2)  
Requirement already satisfied: sympy>=1.13.3 in  
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from  
torch) (1.13.3)  
Requirement already satisfied: networkx in  
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from  
torch) (3.4.2)

Requirement already satisfied: jinja2 in  
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from  
torch) (3.1.6)

Requirement already satisfied: fsspec in  
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from  
torch) (2025.2.0)

Requirement already satisfied: setuptools in  
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from  
torch) (75.8.0)

Requirement already satisfied: farama-notifications>=0.0.1 in  
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from  
gymnasium<1.2.0,>=0.29.1->stable-baselines3) (0.0.4)

Requirement already satisfied: contourpy>=1.0.1 in  
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from  
matplotlib) (1.3.1)

Requirement already satisfied: cycler>=0.10 in  
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from  
matplotlib) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in  
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from  
matplotlib) (4.56.0)

Requirement already satisfied: kiwisolver>=1.3.1 in  
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from  
matplotlib) (1.4.8)

Requirement already satisfied: packaging>=20.0 in  
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from  
matplotlib) (24.2)

Requirement already satisfied: pillow>=8 in  
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from  
matplotlib) (11.1.0)

Requirement already satisfied: pyparsing>=2.3.1 in  
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from  
matplotlib) (3.2.1)

Requirement already satisfied: python-dateutil>=2.7 in  
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from  
matplotlib) (2.9.0.post0)

Requirement already satisfied: six>=1.5 in  
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from  
python-dateutil>=2.7->matplotlib) (1.17.0)

Requirement already satisfied: mpmath<1.4,>=1.1.0 in  
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from  
sympy>=1.13.3->torch) (1.3.0)

Requirement already satisfied: MarkupSafe>=2.0 in  
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from  
jinja2->torch) (3.0.2)

Requirement already satisfied: pytz>=2020.1 in  
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from  
pandas->stable-baselines3) (2024.2)

Requirement already satisfied: tzdata>=2022.7 in  
c:\users\coren\appdata\local\programs\python\python313\lib\site-packages (from  
pandas->stable-baselines3) (2025.1)

### 2.1.1 Test imports

```
[4]: from javascript import require, On, Once, AsyncTask, once, off
import time, math, os
import numpy as np
import gym
from gym import spaces
```

[JSE] (node:7692) [DEP0040] DeprecationWarning: The `punycode` module is deprecated. Please use a userland alternative instead.

[JSE] (Use `node --trace-deprecation ...` to show where the warning was created)

## 2.2 Bot Connection Helper

Define a utility function to spawn and login the Mineflayer bot:

- Explain that this handles network connection and event waiting

We import Mineflayer & Vec3 (Node.js libs exposed via javascript package)

```
[5]: mineflayer = require('mineflayer')
Vec3 = require('vec3').Vec3
```

We define a function to spawn and login the Agent to the server in localhost.

```
[6]: def create_bot(host='localhost', port=25565, username="N7JumpingBot"):
    """Spawn Mineflayer bot and wait for login event."""
    bot = mineflayer.createBot({
        'host': host,
        'port': port,
        'username': username,
        'hideErrors': False
    })
    once(bot, 'login')
    print(f' Logged in as {username}')
    return bot
```

## 2.3 Gym-Style Environment Class

Wrap the Mineflayer bot as a Gym Env:

- Describe observation\_space (9×6 block matrix)
- Describe action\_space (MultiBinary 6 for movement controls)
- List the methods you'll implement: `__init__`, `get_visible_blocks`, `reset`, `step`, `_compute_reward`, `_check_done`, `close`

```

[7]: class MinecraftRL(gym.Env):
    metadata = {'render.modes': ['human']}

    def __init__(self, host='localhost', port=25565, turn_delta=0.15):
        super().__init__()
        self.bot = create_bot(host, port)
        # 9 heights x 6 depths
        self.observation_space = spaces.Box(low=0, high=4096, shape=(9,6),
        dtype=np.int32)
        # forward, back, strafe_left, strafe_right, jump, sprint, turn_left,
        turn_right
        self.action_space = spaces.MultiBinary(8)
        self.turn_delta = turn_delta

    def get_visible_blocks(self):
        """Returns (9x6) matrix of block.type in front of the agent."""
        yaw = self.bot.entity.yaw
        dx, dz = -math.sin(yaw), math.cos(yaw)
        px, py, pz = (self.bot.entity.position.x,
                      self.bot.entity.position.y,
                      self.bot.entity.position.z)
        vis = np.zeros((9,6), dtype=np.int32)
        for d in range(1,7):
            bx = int(round(px + dx*d))
            bz = int(round(pz + dz*d))
            for i, h in enumerate(range(-4,5)):
                by = int(math.floor(py + h))
                block = self.bot.blockAt(Vec3(bx,by,bz))
                vis[i, d-1] = block.type if block else 0
        return vis

    def reset(self):
        """Teleport or respawn logic goes here; then return initial obs."""
        # e.g. self.bot.chat('/tp ...') or custom plugin call
        time.sleep(0.5)
        return self.get_visible_blocks()

    def step(self, action):
        """
        action: [fwd, back, strafeL, strafeR, jump, sprint, turnL, turnR]
        """
        controls = ['forward', 'back', 'left', 'right', 'jump', 'sprint']
        # apply movement controls
        for idx, name in enumerate(controls):
            self.bot.setControlState(name, bool(action[idx]))
        # apply turning by directly adjusting yaw
        yaw = self.bot.entity.yaw

```

```

        if action[6]:
            yaw += self.turn_delta
        if action[7]:
            yaw -= self.turn_delta
        self.bot.look(yaw, self.bot.entity.pitch)
        time.sleep(0.1)

        obs = self.get_visible_blocks()
        reward = self._compute_reward(obs)
        done = self._check_done(obs)
        return obs, reward, done, {}

    def _compute_reward(self, obs):
        """TODO: reward."""
        # -1 when advancing, -100 for falling, +100 for success (maybe adapting
        ↪ it compared to the distance to the finish may be better)
        # Maybe add 50 for reaching some "check point"
        return -1

    def _check_done(self, obs):
        """TODO: use the diamond block platform"""
        return False

    def close(self):
        """Cleanly disconnect."""
        self.bot.quit()

```

## 2.4 Observation Utility Functions

Break out any helper functions used inside your Env, such as:

- Converting world coordinates to agent-relative indices
- Sampling direction vectors from yaw

```

[8]: def world_to_local(px, py, pz, yaw, depth, height_off):
        """Convert a (depth, height) in front-of-agent into world Vec3 coords."""
        dx, dz = -math.sin(yaw), math.cos(yaw)
        x = int(round(px + dx*depth))
        y = int(math.floor(py + height_off))
        z = int(round(pz + dz*depth))
        return Vec3(x,y,z)

    def get_direction_vector(yaw):
        """Returns unit-vector (dx, dz) for facing direction."""
        return (-math.sin(yaw), math.cos(yaw))
    # Example reward & done stubs-tailor to your map:
    GOAL_BLOCK_ID = 57 # e.g. gold pressure plate at finish

```

```
def compute_progress_reward(prev_vis, curr_vis):
    """Example: reward +1 for seeing more goal-blocks in front."""
    return float((curr_vis == GOAL_BLOCK_ID).sum() - (prev_vis ==
    ↪GOAL_BLOCK_ID).sum())

def check_fall_done(bot):
    """Example: done if Y < 0."""
    return bot.entity.position.y < 0
```

## 2.5 Reward & Done Logic

Outline your reward shaping and terminal conditions:

- Progress reward (e.g. moving closer to goal)
- Penalty for falling or exceeding time
- Done flag when finish line reached or failure

```
[9]: # Example reward & done stubs—tailor to your map:
GOAL_BLOCK_ID = 57 # e.g. gold pressure plate at finish

def compute_progress_reward(prev_vis, curr_vis):
    """Example: reward +1 for seeing more goal-blocks in front."""
    return float((curr_vis == GOAL_BLOCK_ID).sum() - (prev_vis ==
    ↪GOAL_BLOCK_ID).sum())

def check_fall_done(bot):
    """Example: done if Y < 0."""
    return bot.entity.position.y < 0
```

## 2.6 Visualization Helpers

Provide functions to help you debug and visualize:

- Plotting the block-ID matrix as a heatmap
- Rendering a simple textual or graphical view of the agent's surroundings

```
[10]: import matplotlib.pyplot as plt

def plot_block_matrix(vis):
    """Heatmap of block IDs (default colormap)."""
    plt.figure()
    plt.imshow(vis, origin='lower', aspect='auto')
    plt.colorbar(label='Block ID')
    plt.xlabel('Depth (blocks ahead)')
    plt.ylabel('Height offset (-4..+4)')
    plt.title('Visible Blocks')
    plt.show()

def render_agent_view(env):
    """Quick render of the agent's current visible blocks."""
```

```
vis = env.get_visible_blocks()
plot_block_matrix(vis)
```

## 2.7 Wrapping with Stable-Baselines3

Explain how to instantiate an RL agent:

- Choose PPO (or DQN)
- Pass your MinecraftRL env into the model

```
[11]: from stable_baselines3 import PPO
from stable_baselines3.common.monitor import Monitor

log_dir = "./logs/"
os.makedirs(log_dir, exist_ok=True)
monitored_env = Monitor(MinecraftRL(), log_dir)

model = PPO("MlpPolicy", monitored_env, verbose=1)
```

Logged in as N7ParkourBot

-----  
AssertionError Traceback (most recent call last)

Cell In[11], line 6

```
4 log_dir = "./logs/"
5 os.makedirs(log_dir, exist_ok=True)
----> 6 monitored_env = Monitor(MinecraftRL(), log_dir)
8 model = PPO("MlpPolicy", monitored_env, verbose=1)
```

File

```
~\AppData\Local\Programs\Python\Python313\Lib\site-packages\stable_baselines3\common\monit
py:40, in Monitor.__init__(self, env, filename, allow_early_resets,
reset_keywords, info_keywords, override_existing)
31 def __init__(
32     self,
33     env: gym.Env,
34     filename: str,
35     allow_early_resets: bool = False,
36     reset_keywords: List[str] = None,
37     info_keywords: List[str] = None,
38     override_existing: bool = True,
39 ):
----> 40     super().__init__(env=env)
41     self.t_start = time.time()
42     self.results_writer = None
```

File ~\AppData\Local\Programs\Python\Python313\Lib\site-packages\gymnasium\core

```
py:313, in Wrapper.__init__(self, env)
307 """Wraps an environment to allow a modular transformation of the :meth:
308 `step` and :meth:`reset` methods.
309 Args:
310     env: The environment to wrap
```



```

311 """
312 self.env = env
--> 313 assert isinstance(
314     env, Env
315 ), f"Expected env to be a `gymnasium.Env` but got {type(env)}"
317 self._action_space: spaces.Space[WrapperActType] | None = None
318 self._observation_space: spaces.Space[WrapperObsType] | None = None

AssertionError: Expected env to be a `gymnasium.Env` but got <class '__main__.
↳MinecraftRL'>

```

## 2.8 Training Loop

Describe running `model.learn`:

- Total timesteps or episodes
- Where you log rewards/metrics

```

[2]: from stable_baselines3.common.callbacks import CheckpointCallback

# save every 10k steps
checkpoint_cb = CheckpointCallback(save_freq=10_000, save_path=log_dir,
↳name_prefix="ppo_mc")
model.learn(total_timesteps=200_000, callback=checkpoint_cb)

```

```

-----
NameError                                Traceback (most recent call last)
Cell In[2], line 4
      1 from stable_baselines3.common.callbacks import CheckpointCallback
      3 # save every 10k steps
----> 4 checkpoint_cb = CheckpointCallback(save_freq=10_000, save_path=log_dir,
↳name_prefix="ppo_mc")
      5 model.learn(total_timesteps=200_000, callback=checkpoint_cb)

NameError: name 'log_dir' is not defined

```

## 2.9 Training Logs & Plots

Show how to load or collect your training logs and plot the learning curve:

- Use Matplotlib to plot episode returns over time

```

[3]: import pandas as pd

df = pd.read_csv(os.path.join(log_dir, "monitor.csv"), skiprows=1)
# rolling mean of reward
df['r_mean'] = df['r'].rolling(10).mean()

```

```
plt.figure()
plt.plot(df['r_mean'])
plt.xlabel("Episode")
plt.ylabel("Rolling Mean Reward")
plt.show()
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[3], line 3
      1 import pandas as pd
----> 3 df = pd.read_csv(os.path.join(log_dir, "monitor.csv"), skiprows=1)
      4 # rolling mean of reward
      5 df['r_mean'] = df['r'].rolling(10).mean()

NameError: name 'os' is not defined
```

## 2.10 Evaluation & Demo Episodes

Roll out a few episodes under the trained policy:

- Reset env, step with `model.predict`
- Use your visualization helpers to display agent behavior

```
[4]: from stable_baselines3.common.evaluation import evaluate_policy

# Automated eval
mean_reward, std_reward = evaluate_policy(model, monitored_env,
    ↪n_eval_episodes=5)
print(f"Mean reward: {mean_reward:.2f} +/- {std_reward:.2f}")

# Manual demo
for ep in range(3):
    obs = monitored_env.reset()
    done = False
    while not done:
        action, _ = model.predict(obs, deterministic=True)
        obs, reward, done, _ = monitored_env.step(action)
        render_agent_view(monitored_env)
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[4], line 4
      1 from stable_baselines3.common.evaluation import evaluate_policy
      3 # Automated eval
----> 4 mean_reward, std_reward = evaluate_policy(model, monitored_env,
    ↪n_eval_episodes=5)
      5 print(f"Mean reward: {mean_reward:.2f} +/- {std_reward:.2f}")
```

```
7 # Manual demo
```

```
NameError: name 'model' is not defined
```

## 2.11 Hyperparameter Tuning Tips

List important hyperparameters to experiment with:

- Learning rate, gamma, clip range, batch size, etc.
- Suggest grid-search or Bayesian tuning approaches

```
[5]: param_grid = {  
    'learning_rate': [1e-4, 3e-4, 1e-3],  
    'gamma': [0.95, 0.99, 0.999],  
    'n_steps': [128, 256, 512],  
}  
print("Example grid:", param_grid)
```

```
Example grid: {'learning_rate': [0.0001, 0.0003, 0.001], 'gamma': [0.95, 0.99,  
0.999], 'n_steps': [128, 256, 512]}
```

## 2.12 Conclusion & Next Steps

Summarize results and propose extensions:

- Continuous yaw control
- Curriculum learning on progressively harder course
- Integration with more complex objectives like doing courses using different mechanics like using a ladder on the side of a block, having an obstacle moving on a jump trajectory and more complex blocks arrangements needing to guess the good way when there are many possible next jump choice possible.